

## El trabajo práctico a elaborar se centra en el Desarrollo de Suite de Juegos "Azar y Lógica" en Python

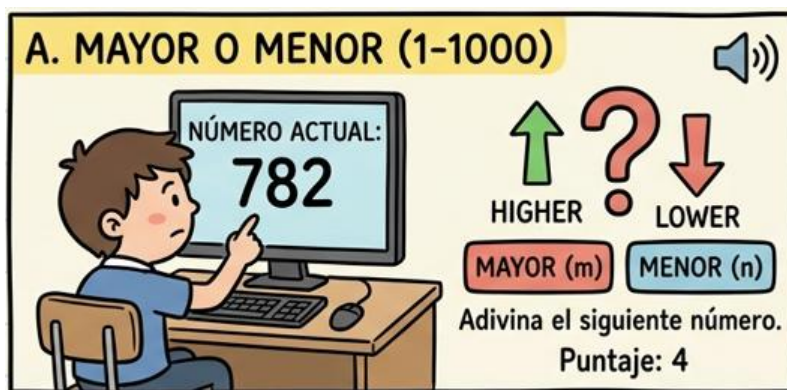
### 1. Introducción

El objetivo de este trabajo es aplicar los conceptos de **programación estructurada** y **diseño descendente** mediante la creación de un software que agrupe diversos juegos de mesa y azar para disfrutar con amigos y familiares.

### 2. Generalidades de los juegos a desarrollar.

El programa debe presentar un **Menú Principal** que permita al usuario seleccionar una de las siguientes opciones:

#### A. Juego: Mayor o Menor



El sistema genera un número aleatorio inicial entre 1 y 1000. El usuario debe arriesgar si el siguiente número será **Mayor** o **Menor**.

- Si acierta, suma un punto y el nuevo número pasa a ser el número de referencia.
- El juego termina cuando el usuario falla. Se debe mostrar la racha de aciertos conseguida.

#### B. Juego: El Número Secreto (Adivinar)

La computadora "piensa" un número entre 1 y 100. El usuario tiene un número **máximo de intentos** para adivinarlo.



- En cada intento fallido, el sistema debe indicar si el número secreto es "mayor" o "menor" al ingresado por el usuario.
- El juego termina si el usuario adivina o si se agotan los intentos.

**C. Juego: BlackJack Simple (21)**

- Se reparten 2 cartas a la computadora y a los jugadores.
- El jugador pide "cartas" (números aleatorios del 1 al 10).
- El jugador puede decidir "Plantarse" o "Pedir otra".
- Si el jugador supera 21, pierde automáticamente.
- Si se planta, la casa genera sus propios números. Gana quien esté más cerca de 21 sin pasarse.

Nota: Más adelante, en las especificaciones de desarrollo del juego se mostrarán las reglas completas.

**D. Juego: Dados (Par o Impar)**

El usuario apuesta una cantidad de "créditos" virtuales a que la suma de dos dados será **Par** o **Impar**.

- Se generan dos números aleatorios del 1 al 6.
- Se informa el resultado y se actualiza el saldo de créditos del jugador.

**3. Criterios de Evaluación**

Se tendrá en cuenta:

1. **Validación de Datos:** El programa no debe cerrarse abruptamente. El programa deberá validar todas las entradas del usuario. Ante un dato inválido (opción inexistente, valor fuera de rango o tipo incorrecto), deberá

mostrar un mensaje de error y solicitar nuevamente el ingreso, sin finalizar la ejecución.

2. **Legibilidad del Código:** Uso de nombres de variables descriptivos (ej: puntuación máxima en lugar de p1).
3. **Interfaz de Usuario:** Mensajes claros que guíen al jugador en cada paso, sin errores de ortografía.
4. **Lógica:** Correcta implementación de la lógica de los juegos.
5. **Estética:** Se valorará mucho la estética de presentación, colores, carteles dibujados con caracteres, recuadros, etc.

#### 4. Formato y reglas de Entrega

- El código debe entregarse en un único archivo con extensión .py.
- El nombre del .py debe contener el nombre de los interesados, por ejemplo: "TP1\_PotterHarry\_GrangerHermione\_WesleyRon\_LongbottomNeville.py"
- Dentro del archivo, a modo de comentario, debe incluir un encabezado con los nombres de los integrantes.
- No debe realizarse los diagramas de chapin.
- La programación debe respetar el paradigma de programación estructurada.
- Será un plus la estética y la amigabilidad con el usuario.
- Los grupos deben ser de 4 integrantes.
- Todos tienen que entregar el trabajo en el cvg en el área dispuesta a tal efecto. Aquel alumno que no entregue el trabajo no será calificado aunque el resto de sus compañeros de grupo lo haya entregado. Por supuesto, el trabajo tiene que ser el mismo de todos los integrantes del grupo.
- No se aceptan, en ninguna circunstancia ni ninguna excusa, un trabajo luego de la fecha/hora de cierre de entrega en cvg.
- El trabajo debe ser defendido como lo indiquen los profesores de cada comisión
- Leer el archivo de preguntas frecuentes.